

# Project proposal

Antoine Vitupier, Marcus Hamelink, Antoine Reynaud,  
Ferdinand Chiu, Ilias Sokolis, Noa Duron

March 2026

## 1 Description

The FaceHugger Robot is a 12-degree-of-freedom legged robot, built to mimic the posture and movement of a four-legged mammal. With four legs each driven by three servo motors, it can shift its weight, adjust its stance, and walk in any direction while keeping its body level and stable.

Movement is governed by an 80/20 hybrid model. Eighty percent of the motion comes from hardcoded elliptical foot trajectories that produce a fluid trot gait, giving the robot full 360-degree translation: forward, backward, and lateral strafing in any direction. The remaining twenty percent is a real-time stabilisation layer driven by the IMU. A PID controller reads the current pitch and roll and computes a corrective vertical offset  $\Delta z$  that is applied to all four foot targets simultaneously:

$$z_{\text{actual}} = z_{\text{gait}} + \Delta z_{\text{PID}}$$

This keeps the chassis level over uneven terrain without the gait engine needing any knowledge of the floor geometry.

Underlying both layers is an analytical Inverse Kinematics solver. Every foot target is expressed in a global body frame centred on the robot's centre of mass, so a single body-tilt correction automatically propagates to all four legs at once. This is what gives the robot its characteristic organic appearance: the body floats steadily above shifting feet rather than moving rigidly as one piece.

Beyond standard locomotion, the robot supports a signature maneuver: a self-righting wall-flip. When the front ToF sensor detects a wall at close range, the robot can execute a pre-timed open-loop thrust sequence to push off the surface, rotate in the air, and land upright. The IMU monitors the fall and re-engages normal operation automatically once the robot is stable.

### Primary Objectives

1. **Stand and self-support.** The robot maintains a stable stance under its own weight with all servos holding position.
2. **Teleoperated walking.** A user controls the robot in real time from a smartphone joystick, going forward, backward, lateral, and turning, using the hybrid 80/20 trot gait described above.
3. **Command API.** All motion primitives are accessible via a documented WebSocket/JSON interface, enabling programmatic control from any networked device.

## Extensions

**Bluetooth Person-Following.** A Bluetooth beacon worn by the operator (e.g. a second ESP32) lets the robot estimate the bearer's relative direction from RSSI measurements. The robot turns autonomously to track the beacon without requiring a camera or computer vision.

**Self-Righting Wall-Flip.** When the front-facing ToF sensor detects a wall within a threshold distance and the operator issues the "flip" command, the FSM enters the ACTION state, PID stabilisation is suspended, and Core 0 executes a pre-timed open-loop burst of maximum-torque PWM signals to push the robot off the wall. The IMU monitors orientation and re-engages normal operation once tilt returns below  $10^\circ$ .

### 1.1 Hardware

The hardware architecture is carefully designed to meet the project's requirements, particularly the high current demand, which can reach up to **24A** if all servos operate simultaneously at max current (which is highly unlikely). In the beginning we thought of implementing a distributed power architecture where one DC DC step down buck converter would be used in series with one PCA 9685 PWM multiplexer in each pair of legs to split the current between two pair of legs (12A max each for each pair of legs) and another DC DC step down buck converter to get a 5V output voltage for the micro controller and sensors.

After talking with the professor and the T.As we deemed that this was unnecessary (as the servos will never operate at max current simultaneously) so we proceeded to go with one PCA 9685 PWM multiplexer directly getting power from the battery (even if the servos are rated to work in the range of 5V-7.2V the professor told us that it will likely be ok to operate it like that) as well as DC DC buck converter to step down the voltage for the microcontroller and sensors.

#### 1.1.1 Leg movement

Since we decided to have 3 joints on each leg, we needed to have a total of 12 powerful servos. After carefully reading other projects similar to ours in the past and talking with our T.A. we concluded that the classic SG90 servos do not comfortably supply enough torque.

We proceeded with the **DSS 15Kg\*cm 270°** servos because of their rotation versatility, light weight (70g) and powerful torque. Even if they require a lot of current, our battery will be able to comfortably supply it.

Those servos will receive their signals via a PCA 9685 PWM multiplexer and their power directly from the battery connected to the Vin pin of the PCA9685 PWM multiplexer.

The only problem we can think about with this is that when the battery is fully charged (around 8V) it surpasses the maximum recommended operating voltage of the servos, but the professor told us it should not be a problem, if it is we will just use a DC-DC buck converter that supports large currents (around 10-15A) to step down the voltage of the battery.

#### 1.1.2 Torque Analysis

We selected high-torque servos, so that body weight does not become a binding design constraint. To quantify the actual limits this gives us, we invert the static torque equations and solve for the **maximum body weight** the leg can support as a function of link geometry and joint angles. We model a single leg as a 3-link serial chain rotating in the sagittal (pitch) plane.

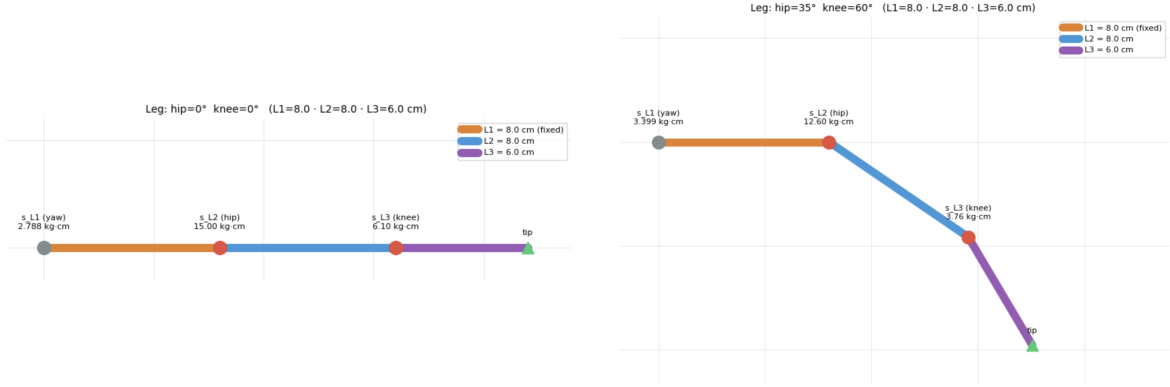


Figure 1: 3-link leg model in extended (left) and bent (right) configurations, illustrating the joint angles and link lengths used in the torque analysis.

**Leg model:** Each leg consists of three links:  $L_1$  (shoulder to hip joint, length  $r_1$ ),  $L_2$  (hip to knee, length  $r_2$ ), and  $L_3$  (knee to foot tip, length  $r_3$ ). The hip and knee motors act on pitch axes and bear static gravity torque, while the shoulder motor acts on a yaw axis (parallel to gravity) and bears only dynamic rotational inertia during swing phase.

**Knee motor torque:** The knee servo supports link  $L_3$  and the tip load  $m_{\text{tip}}$ . All relevant masses lie along  $L_3$ , so a single angle  $\theta_{\text{knee}}$  governs the gravity projection:

$$\tau_{L_3} = \left( m_{L_3} \cdot \frac{r_3}{2} + m_{\text{tip}} \cdot r_3 \right) \cos \theta_{\text{knee}}$$

**Hip motor torque:** The hip servo supports  $L_2$ , the knee servo mass,  $L_3$ , and the tip load. Because the hip and knee angles are independent, masses on  $L_2$  project by  $\cos \theta_{\text{hip}}$  while masses on  $L_3$  project by  $\cos \theta_{\text{knee}}$ :

$$\begin{aligned} \tau_{L_2} = & m_{L_2} \frac{r_2}{2} \cos \theta_{\text{hip}} + m_{\text{servo,knee}} r_2 \cos \theta_{\text{hip}} \\ & + m_{L_3} \left( r_2 \cos \theta_{\text{hip}} + \frac{r_3}{2} \cos \theta_{\text{knee}} \right) \\ & + m_{\text{tip}} (r_2 \cos \theta_{\text{hip}} + r_3 \cos \theta_{\text{knee}}) \end{aligned}$$

**Inverse solver to find the weight budget:** With the motor choice settled, the practical question is how much total mass we can allocate to the chassis, electronics, and payload. Setting each torque equal to the rated stall torque and solving for  $m_{\text{tip}}$  yields the per-leg load capacity. For the hip:

$$m_{\text{tip,max}}^{\text{hip}} = \frac{\tau_{\text{rated}} - \left( m_{L_2} \frac{r_2}{2} + m_{\text{servo,knee}} r_2 \right) \cos \theta_{\text{hip}} - m_{L_3} \left( r_2 \cos \theta_{\text{hip}} + \frac{r_3}{2} \cos \theta_{\text{knee}} \right)}{r_2 \cos \theta_{\text{hip}} + r_3 \cos \theta_{\text{knee}}}$$

The binding constraint is whichever servo saturates first. Total supportable body mass is then:

$$m_{\text{body}} = \min(m_{\text{tip,max}}^{\text{knee}}, m_{\text{tip,max}}^{\text{hip}}) \times n_{\text{legs in stance}}$$

For link lengths up to 10 cm at worst-case joint angles ( $\theta = 0^\circ$ ), this yields a minimum supportable body weight of 3–4 kg across four legs, comfortably above our target chassis mass.

**Dynamic yaw torque:** As a secondary analysis, we compute the moment of inertia of the fully extended leg about the shoulder yaw axis (using the parallel-axis theorem for each link and point mass) and derive the required yaw torque  $\tau_{\text{yaw}} = I_{\text{leg}} \cdot \alpha$  for a target swing-phase angular acceleration

$\alpha$ . Checking the simulation across the full parameter range confirms comfortable margins for any realistic leg configuration given the 15 kg cm rated torque of our servos.

An interactive Jupyter notebook (`torque_calculations.ipynb`) allows exploration of these equations across the full range of joint angles and design parameters.

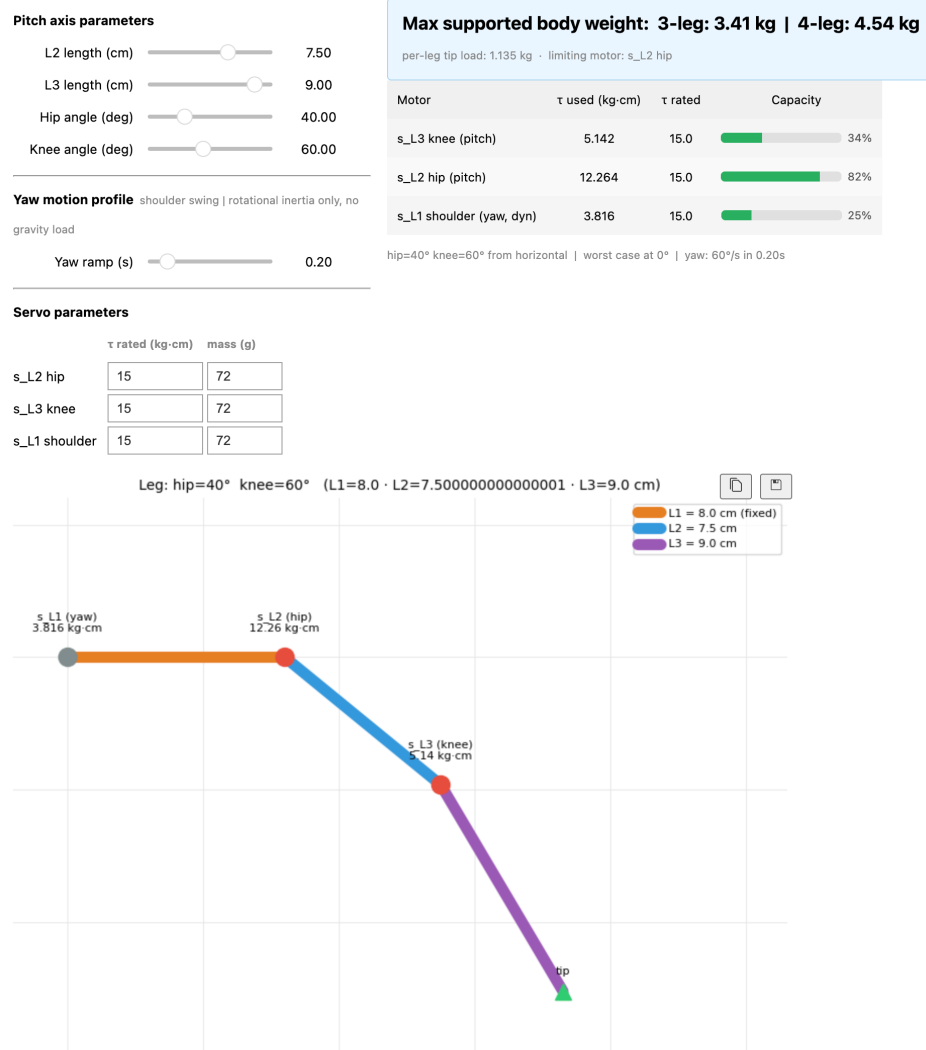


Figure 2: Screenshot of the interactive Jupyter notebook widget, which allows exploration of the torque equations across the full range of joint angles and design parameters.

### 1.1.3 Central Processing Unit

For the brain of our robot, we chose an **ESP32 Devkit V1 Wifi/Bluetooth Dual Core micro controller**. The dual core setup enables us to divide our software architecture in two parts (see below): core 1 handles the **JSON parsing of the packets** coming from our remote control app while core 2 handles the **nervous system** of the robot (inverse kinematics engine). The **Wifi/Bluetooth** functionality enables us to **remotely control** the robot via our Command API and the numerous ports enable us to equip our robot with the sensors required via an **I2C multiplexer**.

#### 1.1.4 Sensors

For the senses of the FaceHugger, we chose to give it an **accelerometer + gyroscope** sensor which is essential to understanding the robot's current orientation and speed in order to be able to maneuver more complex terrains such as stairs while keeping the equilibrium. We also decided to equip it with 5 time of flight distance sensors (**VL53L0X (ToF) I2C Laser Distance Sensor**) for the front, left, right, back and bottom part of robot, in order to be able to detect its surroundings if we want to extend it to be more autonomous. The sensors will get power via the 3.3V voltage output pin of the ESP32 microcontroller.

We also decided to add a small **SBC-OLED screen** in order to give a "face" to our robot.

#### 1.1.5 Power & Power Management

For the heartbeat of our robot we chose a **2S 1300 mAh 30C (39A max) LiPo battery** connected in series with a battery protection module as well as an on/off switch to switch off/on the robot to which we will connect 1 buck converter and one PCA 9685 PWM multiplexer in parallel:

- **LM2596 DC-DC Step-Down Converter Module with Display** to connect to the ESP32 controller (stepped down to 5V) that will supply power both to the micro controller and the sensors
- **PCA 9685 PWM multiplexer** to connect the servos that will get power directly from the battery

### 1.2 Software

The software architecture is designed to manage the high computational demands of 12-DOF Inverse Kinematics (IK) and real-time stability by leveraging the ESP32's dual-core processor. We utilize a FreeRTOS multitasking environment to ensure that time-critical motor movements are never interrupted by network latency or sensor polling.

#### 1.2.1 Dual-Core Execution Model

To maintain smooth and responsive movement, the software is partitioned into two primary execution loops:

- **Core 0 (Nervous System):** Dedicated to the high-frequency Kinematics loop (50Hz–100Hz). This core calculates joint angles and updates the PCA9685 PWM multiplexer.
- **Core 1 (Brain):** Manages high-level logic, including WiFi/WebSocket communication, JSON packet parsing from the remote controller, and sensor fusion from the IMU and ToF sensors.

#### 1.2.2 Inverse Kinematics (IK) Implementation

Inverse Kinematics is the mathematical process of calculating required joint angles ( $\theta_1, \theta_2, \theta_3$ ) to reach a specific coordinate  $(x, y, z)$ . This is fundamental for achieving smooth and controlled movement without manually programming every servo rotation.

- **Geometric Modeling:** Using measurements from our CAD design, we apply trigonometric identities to map the workspace of each leg.
- **Gait Integration:** IK allows us to define "steps" as parametric elliptical paths, enabling the robot to walk forward, backward, or laterally.
- **Stability:** By decoupling foot position from servo angles, we can dynamically shift the center of mass to maintain balance when standing on fewer than four legs.

### 1.2.3 Control Philosophy: The 80/20 Hybrid Model

Stability is the primary challenge for our quadruped. We implement a hybrid control strategy to balance reliability with environmental reactivity:

- **Open-Loop Trajectories (80%):** The robot follows pre-calculated elliptical paths for forward, backward, and lateral (strafing) gaits.
- **Reactive Stabilization (20%):** A PID Controller uses real-time data from the MPU6050 IMU to calculate body-tilt offsets. These offsets are added to the leg coordinates to keep the chassis level on uneven terrain.

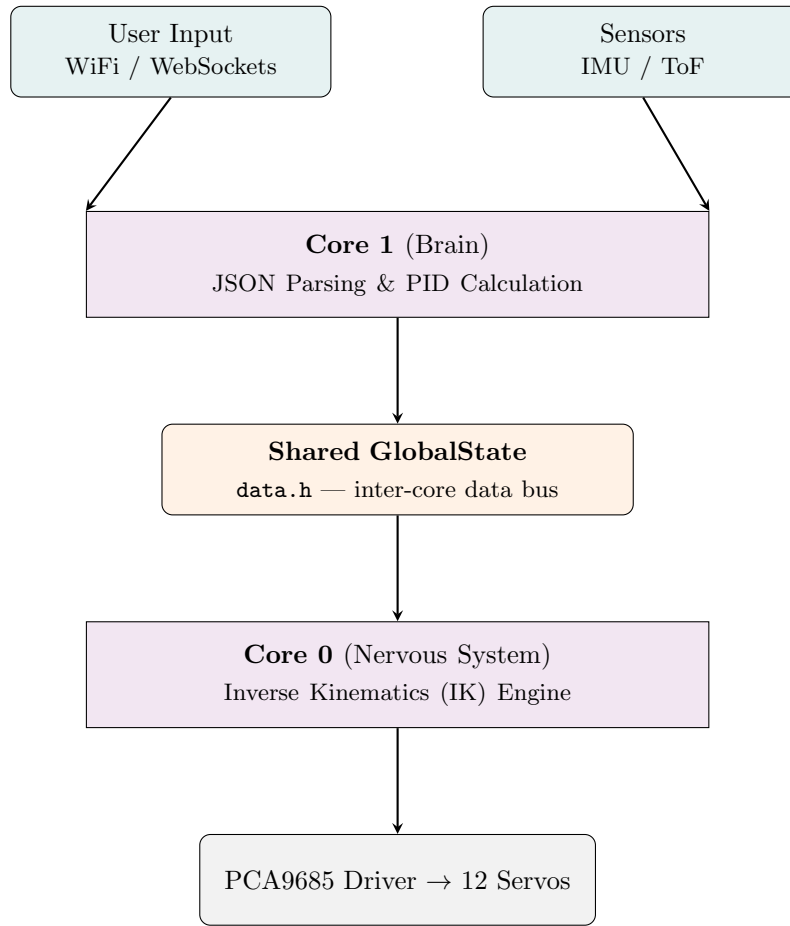
### 1.2.4 Finite State Machine (FSM)

The robot's behavior is governed by a State Machine to ensure safe transitions between maneuvers:

- **IDLE:** Active balancing is enabled; the robot utilizes the PID loop to resist external tilting forces.
- **WALK:** The gait engine is active, merging hardcoded steps with dynamic PID leveling.
- **ACTION:** Specialized sequences, such as the Wall-Flip, are triggered. The ToF sensors verify wall proximity before inhibiting stabilization to allow high-torque rotation.
- **FAILSAFE:** A safety interrupt cuts power to the servos if the battery voltage drops below a certain threshold or if a fall is detected. Precise details

### 1.2.5 Code Architecture Flow

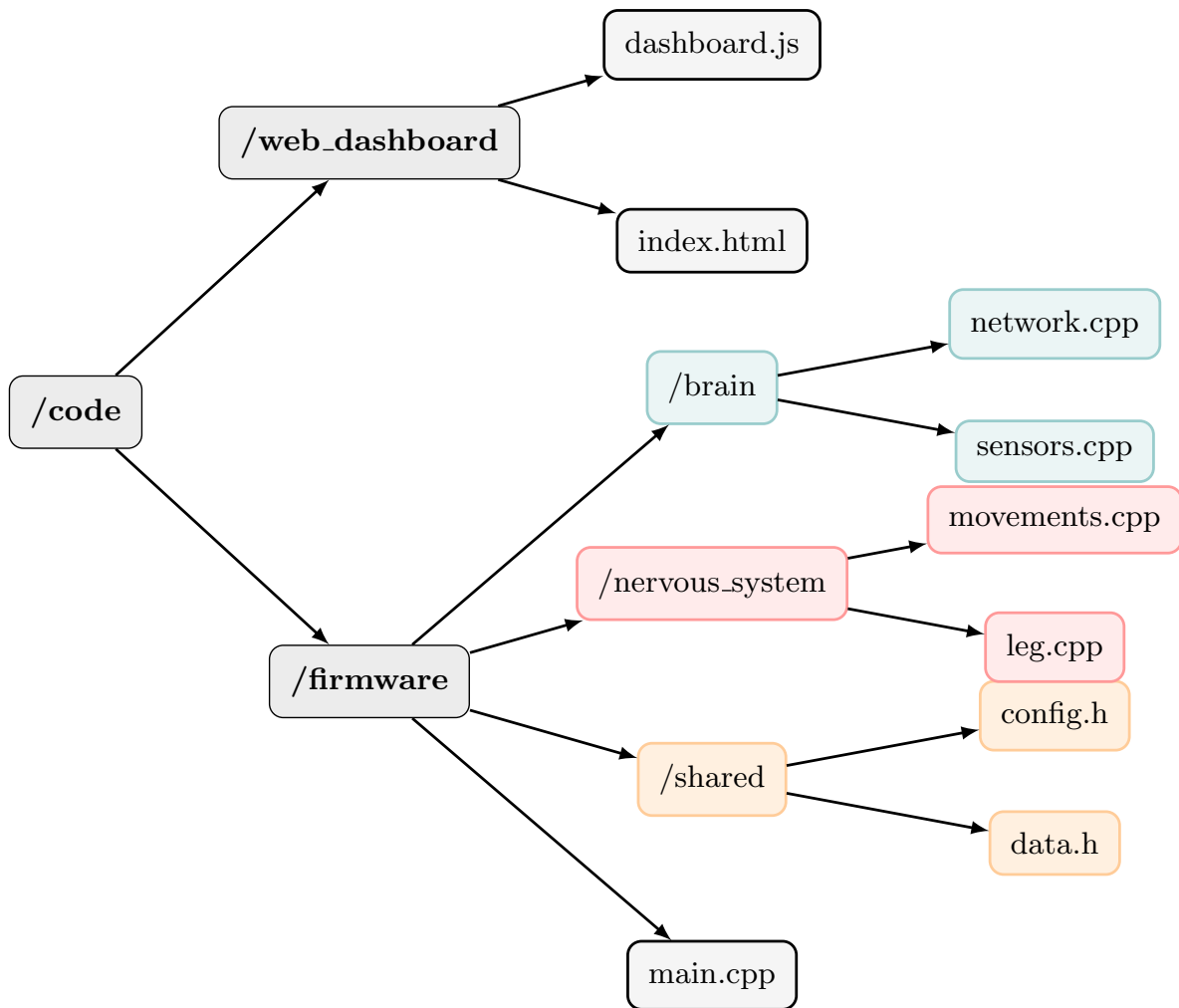
The interaction between the user, the environment, and the dual-core execution is visualized in the following logic flow:



This separation ensures that the **Nervous System** maintains a deterministic refresh rate for the servos, while the **Brain** handles asynchronous tasks like network requests and sensor filtering without causing physical lag in the robot's movement.

### 1.2.6 Repository Partitioning Scheme

The following directory tree illustrates the modular separation of the robot's control systems:



This is how we plan to organize our code for the moment, clearly separating the web dashboard part from the actual implementation of the movement of the quadruped .

### 1.3 Physical Structure

## 2 CAD Design

Manual (8.1, 8.5.2): Complete technical drawings of all structural and mechanical parts, produced in Fusion 360. Must show dimensions so staff can verify parts in the BOM are appropriate.

Formal CAD drawings required | no hand-drawn sketches.

CAD is a living document, updated throughout the semester.

For Ferdinand



## 2.1 Technical drawings

## 2.2 Circuit diagram

# 3 Risk Assessment

### 1. Weight, leg lever-arms, and servo strength (torque margin)

*What could go wrong:* BeetleBot identified weight/servo strength as a main project risk and later reported that early leg designs were too long/heavy, making it hard or impossible to stand/walk without overloading servos; even after optimization, servos still struggled. For our **quadruped**, this risk is higher because each leg carries a larger fraction of the total load and there is less redundancy than a hexapod. If torque margin is insufficient, we will fail to achieve a stable stand and any walking demo.

*Mitigation (prototype + when):* By **Week 6**, prototype a **single-leg + hip load test** (or 2-leg front module) to validate stall/hold performance at the worst joint angles. We will explicitly design for short lever arms (legs closer under the body), keep heavy components low/central, and reduce printed mass (cutouts, low infill where safe). If results are marginal, we will (i) reduce leg length/stance height and (ii) upgrade only the most loaded joints to higher-torque servos before procurement is locked.

### 2. LiPo power delivery and brownouts during peak servo load

*What could go wrong:* BeetleBot flagged LiPo + regulation as risky: many servos moving together can cause voltage drops, resets, and servo glitches. Our system similarly uses a single battery with a 5V rail feeding many servos (via PCA9685) plus the ESP32; sudden current peaks (startup/stall) can reset the ESP32 or cause unstable behavior that appears as a software issue.

*Mitigation (prototype + when):* By **Week 6–7**, bench-test the full power chain (LiPo → buck → servo rail + logic rail) with worst-case motion patterns. Use thick wiring for the servo rail, star grounding, and bulk capacitance near the servo supply. Implement software-side current shaping: avoid commanding all joints to step simultaneously (ramping/interpolation) and enforce conservative acceleration/step height. If necessary, split rails (separate regulator for ESP32) or keep an optional second battery plan.

### 3. Servo coordination, timing, and smoothness (control loop reliability)

*What could go wrong:* BeetleBot explicitly noted that coordinating many servos is challenging (timing, synchronization, responsiveness). For our quadruped, control jitter or blocking code will directly translate into instability (falls) and will slow tuning significantly.

*Mitigation (prototype + when):* Use a fixed-rate motion update loop on ESP32 (target 50–100 Hz) and keep time-critical motor updates isolated from network and sensor tasks (FreeRTOS priorities / dual-core split). Start with **basic primitives** (stand, shift weight, lift one foot) before full gaits, and use interpolation/ramping for smooth trajectories.

### 4. IK implementation sensitivity to real-world measurements and calibration

*What could go wrong:* BeetleBot’s proposal and README both show IK is measurement-sensitive: small differences in link lengths/servo positions can produce inaccurate motion, and tuning can become time-consuming late in the project. If our CAD measurements do not match the built robot, gaits will drift, legs may bind, and stability control will be unreliable.

*Mitigation (prototype + when):* By **Week 7–8**, implement a calibration mode: center servos, store per-servo offsets, enforce joint limits, and verify foot positions on a test stand. Measure real link lengths and update IK constants accordingly. Maintain a fallback “hardcoded stance / crawl” mode (like BeetleBot’s compromise approach) so integration/testing can proceed even if IK is still being refined.

5. **Gait stability for a quadruped (risk of spending too long on dynamic gaits)**

*What could go wrong:* BeetleBot emphasized stability and balance (center of mass must remain within support polygon) and planned both static and dynamic stability work. A quadruped has tighter stability constraints than a hexapod; attempting dynamic gaits (trot/dance) too early can lead to repeated failures and jeopardize the demo.

*Mitigation (prototype + when):* By **Week 8–9**, target a robust **static crawl** (3 feet down) and stable pose transitions (sit/stand). Only after this baseline works will we attempt faster gaits as **optional extensions**. PID-based leveling (IMU) will be introduced progressively: first validate open-loop crawl, then add stabilization and re-tune step sizes.

6. **Remote control over WiFi/WebSocket: integration complexity and coupling to motion control**

*What could go wrong:* BeetleBot identified remote control as an additional complexity layer and noted that WiFi/Web APIs require background knowledge; tight coupling between comms and walking logic increases debugging difficulty. For our system, network latency or parsing delays could interfere with control timing if not isolated.

*Mitigation (prototype + when):* Keep communication logic modular and separate from motion control (queue commands from “brain” task to “nervous system” task). Define a minimal protocol (small JSON messages is acceptable if rate-limited). Validate the web interface early on a bench setup (ESP32 + PCA9685 + 1–2 servos). Maintain a serial “manual override” fallback for debugging if WiFi issues appear.

7. **Demount/remount difficulty (maintenance and iteration speed)**

*What could go wrong:* Based on prior legged-robot experience (e.g., BeetleBot’s emphasis on complex cable management and frequent hardware access), a major schedule risk is having a design that is hard to open, service, or reassemble. If servos, the PCA9685, sensors, or the ESP32 are not easily accessible, every failure (servo replacement, wire debugging, re-printing one part, re-routing cables) becomes a long operation. This slows down iteration, increases the chance of wiring mistakes after reassembly, and can block integration/testing late in the project.

*Mitigation (prototype + when):* By **Week 6–7**, enforce a **serviceable mechanical/electrical design**: (i) split the chassis into modules (e.g., leg modules + electronics bay), (ii) use a lid/cover with a small number of standard fasteners and clear access to the PCA9685 and power chain, (iii) add strain relief and tie points so cable bundles keep their shape during opening/closing, (iv) label connectors and define a fixed pin map, (v) time a “maintenance drill” (open body, replace one servo, close body) and iterate CAD until it can be done quickly and repeatably.

## 4 Bill of Materials

For the bill of materials we will need (see Table 1):

## 5 Work Breakdown

| Item                                                                                                       | Qty | Source / Link | Price (CHF) |
|------------------------------------------------------------------------------------------------------------|-----|---------------|-------------|
| ESP32 Devkit V1<br>Wifi/Bluetooth Dual core                                                                | 1   | Link to buy   | 8           |
| PCA9685 PWM Multiplexer                                                                                    | 1   | Link to buy   | 11          |
| 15Kg/cm 270° Servo DSS-M15S with Metal Gear                                                                | 12  | Link to buy   | 145         |
| Li-Po 2S 7,4V 30C 1.300 mAh,<br>contact JST-RCY (BEC)                                                      | 1   | Link to buy   | 20          |
| IMU MPU6050 Accelerometer<br>+ Gyroscope                                                                   | 1   | Link to buy   | 13          |
| VL53L0X (ToF) I2C Laser<br>Distance Sensor                                                                 | 5   | Link to buy   | 30          |
| LM2596 DC-DC Step-Down<br>Converter Module with Dis-<br>play to connect with ESP32<br>controller + sensors | 1   | Link to buy   | 10          |
| Female XT60 connector                                                                                      | 2   | Link to buy   | 2           |
| Male XT60 connector                                                                                        | 2   | Link to buy   | 2           |
| ON/OFF switch                                                                                              | 1   | Link to buy   | 1           |
| I2C multiplexer                                                                                            | 1   | Link to buy   | 5           |
| Battery protection module for<br>lipo 2S 1300 mAh 30C (39A)                                                | 1   | Link to buy   | 10          |
| XT60 Charging cable                                                                                        | 1   | Link to buy   | 5           |
| SBC-OLED01V2 128x64 pixels<br>I2C screen                                                                   | 1   | Link to buy   | 10          |
| <b>Total</b>                                                                                               |     |               | <b>272</b>  |

Table 1: Bill of Materials

## GANTT CHART

Marcus Hamelink, Ferdinand Chiu, Antoine Vitupier, Ilias Sokolis, Noa Duron, Antoine Reynaud

**Smartsheet Tip** The visual timeline of a Gantt chart gives a detailed overview of project tasks and dependencies.

The visual timeline of a Gantt chart gives a detailed overview of project tasks and dependencies.

|      |          |
|------|----------|
| DATE | 16/02/26 |
|------|----------|

[illegible]

## 6 Future of the FaceHugger

The current version of the FaceHugger serves as a high-performance 12-DOF baseline. Future iterations could focus on transitioning the robot from a teleoperated machine to an autonomous, environment-aware agent:

- **Vision-Based Spatial Awareness:** By integrating an **ESP32-CAM** or a small **LiDAR** module, the robot could move beyond simple ToF distance measurements. This would allow for SLAM (Simultaneous Localization and Mapping), enabling the FaceHugger to navigate complex indoor environments and autonomously plan paths around obstacles.
- **Transition to Full Dynamic Stability:** As the gait engine matures, shift the 80/20 hybrid model toward a more reactive model. By utilizing high-frequency IMU feedback and ground-contact sensors in the feet, the robot could achieve "blind" stabilization—navigating stairs or slippery surfaces where hardcoded elliptical paths are insufficient.
- **Terrain Adaptation and Obstacle Scaling:** With the 270° range of the **DSM 15-270** servos, the FaceHugger has the mechanical potential for extreme reach. Future software updates would include a "High-Step" mode, where the robot uses its ToF sensors to measure the height of a ledge and dynamically recalculates its Inverse Kinematics to mount obstacles up to 10cm in height.
- **Edge-AI for Gesture Control:** Utilizing the dual-core capability further, Core 1 could run lightweight Machine Learning models to recognize hand gestures or follow a specific person without the need for a Bluetooth beacon, making the interaction more organic.

## 7 References and Inspirations

### BeetleBot — EPFL CS-358 (2025 Hexapod)

*Source:* <https://github.com/epfl-cs358/2025sp-beetlebot>

*Relevance:* Provided the baseline for the ESP32/PCA9685 wiring topology and highlighted the critical need to upgrade from SG90 to high-torque servos to avoid structural failure.

### ResearchGate — Four-Legged 12-Joint Robot CAD Model *Source:*

[https://www.researchgate.net/figure/fig2\\_384894431](https://www.researchgate.net/figure/fig2_384894431)

*Relevance:* Serves as the primary structural blueprint for our 3-DOF leg assembly, specifically guiding the placement of the abduction servos to maximize lateral stability.

### Petoi OpenCat / Bittle (Quadruped)

*Source:* <https://github.com/PetoiCamp/OpenCatEsp32>

*Relevance:* Validated that a dual-core ESP32 can handle 12-DOF Inverse Kinematics on-chip and served as the primary reference for gait-switching logic (Walk vs. Trot).

### Stanford Pupper (Research Quadruped)

*Source:* <https://github.com/stanfordroboticsclub/StanfordQuadruped>

*Relevance:* Informed our 80/20 hybrid control philosophy, specifically the use of parametric elliptical trajectories for the swing phase and stance-width calculations.

### Sesame Robot (Expressive Quadruped)

*Source:* <https://github.com/dorianborian/sesame-robot>

*Relevance:* Inspired the "emotive" extensions of our project, demonstrating how an OLED face can sync with state-machine transitions to indicate robot "mood" or battery status.

**Instructables Spider Robot (Minimal Quadruped)**

*Source:* <https://www.instructables.com/Spider-Robot-DIY-Quadruped>

*Relevance:* Served as a fundamental proof-of-concept for the I2C communication chain and the basic 8-DOF coordinate mapping for 3D-printed quadruped limbs.